

Joypad sul DMD — collegare un pad PS4 (e compatibili)

Guida rapida per la versione **3.7** del kWGillo DMD Server. Copre solo i controller: collegamento via USB e via Bluetooth, verifica, comandi e risoluzione dei problemi. Per tutto il resto vale il manuale completo.

In breve

Non c'è niente da installare. Il driver del DualShock 4 (`hid-sony`) è già nel kernel di Raspberry Pi OS, e il servizio DMD gira come `root`, quindi non ci sono nemmeno permessi da sistemare su `/dev/input`.

Via	Tempo	Quando conviene
USB	trenta secondi	prova rapida, oppure pad lasciato collegato al DMD
Bluetooth	due minuti, una volta sola	uso normale: dopo il primo accoppiamento basta premere PS

1. Cosa serve

Versione	DMD Server 3.6 o successiva (i giochi), 3.3 o successiva (Doom)
Pad	DualShock 4, oppure qualunque pad che Linux riconosca come joystick — un Nacon NC5169, un pad XInput generico
Per USB	un cavo micro-USB dati , non da sola ricarica
Per Bluetooth	il Bluetooth del Pi acceso; niente adattatori né pacchetti aggiuntivi

Il pad serve a due cose distinte: i **giochi** scritti per il pannello (Breakout, Invaders, Snake, Pongo) e **Doom**. Sono pagine diverse con impostazioni proprie, ma il pad è lo stesso e si riconosce una volta sola.

2. Collegamento via USB

Collega il pad al Pi con un cavo micro-USB dati. Fine: non c'è altro da fare.

Il cavo è la causa più frequente di «non funziona». Molti cavi micro-USB economici hanno solo i fili dell'alimentazione: il pad si ricarica, la barra luminosa si accende, e il Pi non lo vede. Se il pad si ricarica ma non compare nella pagina Giochi, prova un altro cavo prima di cercare altrove.

Vai alla verifica del punto 4.

3. Collegamento via Bluetooth

3.1 Metti il pad in accoppiamento

Con il pad **spento**, tieni premuti insieme **Share** e **PS** per circa cinque secondi.

Barra luminosa	Significato
doppio lampeggio bianco rapido	accoppiamento in corso — è quello che serve
lampeggio arancione lento	è solo in ricarica: il pad non è in accoppiamento
fissa (blu, rossa, verde...)	già connesso a qualcosa

3.2 Accoppialo dal Pi

```
sudo bluetoothctl
power on
agent on
default-agent
scan on
```

Dopo qualche secondo compare una riga come:

```
[NEW] Device A0:AB:51:xx:xx:xx Wireless Controller
```

Copia l'indirizzo e prosegui, sostituendolo al posto di quello di esempio:

```
pair A0:AB:51:xx:xx:xx
trust A0:AB:51:xx:xx:xx
connect A0:AB:51:xx:xx:xx
scan off
exit
```

La riga che conta è `trust`. Senza, il pad si riconnette finché non riavvii il Pi, e dopo un riavvio devi rifare tutto. Con `trust`, alla riaccensione ti basta premere il tasto **PS** e si riaggancia da solo.

Lascia `scan on` acceso il meno possibile: mentre scansiona, il Bluetooth del Pi è occupato e l'accoppiamento può fallire per un attimo di distrazione del controller.

4. Verificare che il sistema lo veda

4.1 Dalla web UI — il modo consigliato

Apri la pagina **Giochi**, riquadro *Tastiera e joystick*. Se il pad è a posto compare per nome:

Pad riconosciuti: **Wireless Controller**

La pagina **Doom** ha lo stesso riquadro. Se lo vede una, lo vede l'altra: la lettura dei dispositivi è la stessa dalla versione 3.6.

4.2 Da riga di comando, se qualcosa non torna

```
cat /proc/bus/input/devices | grep -B4 -A5 "Wireless Controller"
```

Cerca la riga dei gestori:

```
H: Handlers=event3 js0
```

È il `js0` che conta. Il servizio non apre a caso tutto quello che trova in `/dev/input`: chiede al kernel quali dispositivi sono tastiere (gestore `kbd`) e quali joystick (gestore `js`), e apre solo quelli. Un mouse o un sensore di temperatura non vengono nemmeno sfiorati.

Un DualShock 4 si presenta come **tre** dispositivi distinti:

Nome	Gestori	Usato dal DMD
Wireless Controller	event3 js0	sì — pulsanti e levette
Wireless Controller Touchpad	event4 mouse1	no
Wireless Controller Motion Sensors	event5	no

Solo il primo ha il gestore `js`, quindi gli altri due vengono ignorati da soli. Non devi fare niente per escluderli.

4.3 Vedere i tasti mentre li premi

```
sudo evtest /dev/input/event3
```

Premi i tasti: devono comparire eventi `EV_KEY` per i pulsanti ed `EV_ABS` per levette e croce direzionale. Se `evtest` non è installato: `sudo apt install evtest`.

5. I comandi

5.1 Nei giochi (Breakout, Invaders, Snake, Pongo)

Comando	Sul pad
muoversi	levetta sinistra, levetta destra, croce direzionale — anche in verticale, dalla 10.3
sparare / lanciare la palla	X, cerchio, quadrato, R1, R2
uscire dalla partita	Share/Select
scorrere il giro dei giochi	Options/Start (fuori dal Game Boy anche PS)
far cominciare una partita	Options (o il tasto PS)

Dalla 5.4 cerchio spara, e non esce più. Era l'ultimo posto in cui un tasto frontale chiudeva la partita: stanno tutti e quattro sotto le stesse dita mentre si gioca, e uno di loro non può portare via il pannello. Per uscire restano Share/Select e PS, che si premono apposta.

Snake sterza in assoluto. Le quattro direzioni puntano dove dicono: su va su, sinistra va a sinistra. La prima stesura sterzava in relativo — sinistra gira a sinistra rispetto a dove stai andando — perché davo per scontato un cabinato con i due pulsanti del flipper. Quel cabinato non esiste: questo è un pannello su una mensola e chi gioca ha un controller in mano, e un controller ha la croce direzionale. Con quattro direzioni vere lo sterzo relativo non è una soluzione ma un indovinello: premendo sinistra mentre si va in giù il serpente girava a destra, che è l'opposto di quello che dice il pollice.

Pongo si gioca in su e in giù, e in avanti fino a metà del proprio campo: croce direzionale o una delle due levette. Fino alla 10.2 le levette nei giochi contavano solo in orizzontale — Breakout e Invaders non avevano bisogno d'altro — e con Pongo il pollice sulla levetta non avrebbe mosso niente.

Il dietrofront si ignora, e si scarta **al momento di muoversi** e non alla pressione: a trenta fotogrammi al secondo due tasti sfiorati dentro lo stesso trentesimo di secondo farebbero un mezzo giro, e il serpente si morderebbe da solo senza aver percorso una casella.

5.2 In Doom

Comando	Sul pad
camminare, passo laterale	levetta sinistra
girarsi	levetta destra
sparare	X, R1, R2
aprire porte / usare	cerchio, quadrato
correre	L1, L2
mappa	triangolo
menu	L3 (pressione della levetta sinistra)
invio	R3 (pressione della levetta destra)
uscire dalla partita	Share/Select

Options, PS e Share/Select **non** appartengono a Doom: sono globali e li legge il gestore dei giochi, altrimenti una pressione sola farebbe due cose. È il motivo per cui menu e invio stanno sulle levette premute, che non usa nessun altro.

5.3 Nel Game Boy

A partita aperta l'emulatore si tiene tutto, perché Start e Select sono suoi:

Comando	Sul pad
A	X, R1, R2
B	cerchio, quadrato, triangolo, L1, L2
Start	Options/Start, L3
Select	Share/Select, R3
uscire dalla partita	PS

I nomi sulle plastiche cambiano da un pad all'altro, e negli anni qualche versione del kernel ha scambiato triangolo e quadrato: per questo le azioni importanti stanno su più pulsanti. Sparare con R2 e con X non dà fastidio a nessuno, e salva chi ha un pad che si dichiara diversamente.

6. Chi può far cominciare una partita

Questa è una distinzione voluta e vale la pena capirla, perché è la differenza fra un pannello che si comanda e uno che si porta via il display da solo.

Origine	Può cominciare una partita?	Perché
Options / PS sul pad	sì, di serie	un pulsante preciso su un pad che si tiene in mano non si preme per sbaglio
tastiera collegata al DMD	no, di serie	una tastiera lasciata sul mobile si sfiora: un tasto premuto per caso non deve rubare il pannello a metà partita
pulsante Gioca della web UI	sì, sempre	è un gesto esplicito
interruttore in Home Assistant	sì, sempre	idem

Le due caselle stanno nella pagina **Giochi**, riquadro *Tastiera e joystick*:

- *Options sul pad può far cominciare una partita* — accesa di serie;
- *Un tasto può far cominciare una partita* — spenta di serie.

A partita **già aperta** la tastiera comanda comunque il gioco: la restrizione riguarda solo il *cominciare*.

7. Levette: perché funzionano su pad diversi

Un pulsante è premuto o rilasciato e basta. Una levetta no: ha un valore dentro un intervallo, **e l'intervallo cambia da pad a pad**.

Pad	Intervallo dichiarato
DualShock 4	0 ... 255
molti pad da PC (Nacon, XInput generici)	-32768 ... 32767

Il DMD non dà per scontato nessuno dei due: **chiede al kernel** l'intervallo vero di ogni asse, dispositivo per dispositivo. Darne per scontato uno vorrebbe dire che sull'altro la levetta risulta sempre a fondo corsa oppure sempre ferma. È il motivo per cui non serve nessuna taratura quando cambi pad.

La conversione in premuto/rilasciato ha **due soglie diverse**: si preme al 40% della corsa e si rilascia al 28%. Con una sola soglia, una levetta tenuta appena oltre il limite genererebbe una raffica di premuto/rilasciato, e a schermo si vedrebbe come un personaggio che scatta invece di camminare.

8. Se qualcosa non va

Sintomo	Causa probabile	Rimedio
il pad si ricarica ma non compare	cavo micro-USB di sola ricarica	usa un cavo dati
via Bluetooth non compare in <code>scan on</code>	non è in accoppiamento	Share + PS per 5 s, cerca il doppio lampeggio bianco
<code>pair</code> risponde <code>Failed to pair</code>	il pad è già accoppiato altrove	<code>remove A0:AB:...</code> e riparti da <code>pair</code>
funziona, ma dopo un riavvio del Pi no	manca <code>trust</code>	<code>sudo bluetoothctl → trust A0:AB:...</code>
compare in <code>/proc/bus/input/devices</code> ma senza <code>js0</code>	modulo <code>joydev</code> non caricato	vedi sotto
il pad c'è ma il DMD non lo usa	casella <i>Accetta comandi dal joystick</i> spenta	pagina Giochi (o Doom)
Options non fa partire niente	<i>Options può far cominciare una partita</i> spenta	pagina Giochi
la partita si chiude da sola	tempo di inattività (180 s di serie)	pagina Giochi, campo <i>Chiudi la partita dopo</i>

Il caso `joydev`

È l'unico in cui il riconoscimento automatico fallisce. Il pad funziona, il kernel lo vede, ma non gli assegna il gestore `js` perché il modulo non è caricato — e il DMD cerca proprio quello.

```
sudo modprobe joydev
echo joydev | sudo tee -a /etc/modules
```

La seconda riga lo rende permanente al riavvio.

Scorciatoia alternativa: nella pagina Giochi (e in quella Doom) c'è il campo **Joystick (percorso)**. Scrivendoci `/dev/input/event3` il riconoscimento automatico viene saltato del

tutto e il dispositivo si usa così com'è. È anche la via giusta quando hai due pad collegati e vuoi che ne comandi uno solo.

9. Più pad, o pad diversi

Con il campo *Joystick (percorso)* vuoto, il DMD accetta comandi da **tutti** i joystick collegati contemporaneamente. Due pad comandano lo stesso gioco: utile per provare, meno per giocare in due — i giochi attuali sono a un giocatore.

Per limitarlo a uno, scrivi il suo percorso nel campo. I percorsi `/dev/input/eventN` però possono cambiare numero fra un riavvio e l'altro se colleghi le periferiche in ordine diverso: se ti capita, lascia il campo vuoto e scollega l'altro pad.

10. Avviare una partita da Home Assistant

Dalla versione **3.7** ogni gioco ha il suo interruttore MQTT, accanto a quello di Doom. Compiono da soli con MQTT Discovery, se `mqtt.discovery` è acceso:

Entità	Effetto
<code>switch.dmd_gioco_breakout</code>	accende e spegne una partita a Breakout
<code>switch.dmd_gioco_invaders</code>	idem per Invaders
<code>switch.dmd_gioco_snake</code>	idem per Snake
<code>switch.dmd_gioco_pongo</code>	idem per Pongo
<code>switch.dmd_doom</code>	idem per Doom

Sono **mutuamente esclusivi**: la presa del pannello è una sola, quindi accendendone uno gli altri tornano a OFF da soli. E lo stato non viene dalla configurazione ma dalla partita in corso: se si chiude per inattività, l'interruttore in Home Assistant torna a OFF senza che nessuno glielo dica.

Da lì, un pulsante fisico o un comando vocale che avvia una partita è un'automazione di tre righe.
